Qwen3-VL-4B-Instruct 算子报告

1. 模型概述

Qwen3-VL-4B-Instruct 是阿里巴巴 Qwen 团队开发的多模态大语言模型，支持图像理解和对话生成。

- **论文**: https://arxiv.org/abs/2503.14465
- **框架**: Transformers
- **模型路径**: `/home/ubuntu/stephen/02-weight/Qwen3-VL-4B-Instruct/`
- **Tokenizer 路径**: 同模型路径

---

2. 算子架构总览

2.1 输入输出

输入	形状	说明
`pixel_values`	(B, 3, H, W)	图像 tensor
`input_ids`	(B, L)	文本 token id
`attention_mask`	(B, L)	注意力 mask

输出	形状	说明
`logits`	(B, L, V)	词汇表预测概率
`hidden_states`	(B, L, D)	最后一层隐状态

2.2 核心组件

```
输入 → [视觉编码器] → [多模态融合] → [语言解码器] → 输出
         (ViT)       (Projector)      (Qwen3 Decoder)
```

---

3. 核心算子详解

3.1 Qwen3VLForConditionalGeneration

功能: 多模态条件生成模型

代码来源: transformers.models.qwen2_vl.modeling_qwen2_vl

组成:

组件	类型	参数量
`model.visual`	Qwen2VLVisionModel	~374M

组件	类型	参数量
`model.language_model`	Qwen3VLModel	~4B
`lm_head`	Linear	151936×2560

3.2 视觉编码器 (Vision Tower)

代码位置: `modeling_qwen2_vl.py: Qwen2VLVisionModel`

配置 (config.json):

```
{
  "depth": 24,
  "hidden_size": 1024,
  "in_channels": 3,
  "intermediate_size": 4096,
  "num_heads": 16,
  "patch_size": 16,
  "spatial_merge_size": 2
}
```

视觉层结构 (24层 ViT):

算子	输入	输出	参数量
`attn.qkv.weight`	(1024, 1024)	(3072, 1024)	3.1M
`attn.qkv.bias`	(1024)	(3072)	3K
`attn.proj.weight`	(1024, 1024)	(1024, 1024)	1M
`attn.proj.bias`	(1024)	(1024)	1K
`mlp.linear_fc1.weight`	(1024, 1024)	(4096, 1024)	4.2M
`mlp.linear_fc1.bias`	(1024)	(4096)	4K
`mlp.linear_fc2.weight`	(4096, 1024)	(1024, 4096)	4.2M
`mlp.linear_fc2.bias`	(4096)	(1024)	4K
`norm1.weight`	(1024)	(1024)	1K
`norm2.weight`	(1024)	(1024)	1K

每层参数量: ~17M

总视觉参数量: ~374M (24层 × 17M + embedding)

3.3 语言模型 (Language Model)

配置 (config.json):

```json
{
  "hidden_size": 2560,
  "num_hidden_layers": 36,
  "num_attention_heads": 32,
  "num_key_value_heads": 8,
  "intermediate_size": 9728,
  "head_dim": 128,
  "vocab_size": 151936,
  "max_position_embeddings": 262144,
  "rms_norm_eps": 1e-06,
  "rope_theta": 5000000
}
```

每层 Transformer 算子:

算子	Shape	参数量公式	说明
`input_layernorm.weight`	(2560)	2560	Pre-LN
`post_attention_layernorm.weight`	(2560)	2560	Post-LN
`self_attn.q_proj`	(4096, 2560)	4096×2560	Q 投影
`self_attn.k_proj`	(1024, 2560)	1024×2560	K 投影
`self_attn.v_proj`	(1024, 2560)	1024×2560	V 投影
`self_attn.o_proj`	(2560, 4096)	2560×4096	输出投影
`self_attn.q_norm.weight`	(128)	128	Q 归一化
`self_attn.k_norm.weight`	(128)	128	K 归一化
`mlp.gate_proj`	(9728, 2560)	9728×2560	SwiGLU 门
`mlp.up_proj`	(9728, 2560)	9728×2560	SwiGLU 上
`mlp.down_proj`	(2560, 9728)	2560×9728	SwiGLU 下

Attention 算子公式:

- Q 投影: `q = x @ W_Q`, shape: `(B, L, 2560) → (B, L, 32, 128)`
- K 投影: `k = x @ W_K`, shape: `(B, L, 2560) → (B, L, 8, 128)`
- V 投影: `v = x @ W_V`, shape: `(B, L, 2560) → (B, L, 8, 128)`
- RoPE 位置编码: 应用于 Q, K
- 注意力计算: `Attention(Q, K, V) = softmax(QK^T / sqrt(d_k))V`
- O 投影: `o = attn @ W_O`

MLP 算子公式 (SwiGLU):

- `gate = silu(x @ W_gate)`
- `up = x @ W_up`
- `mlp = gate * up @ W_down`

每层参数量:

组件	参数量
Attention (Q,K,V,O)	4.7M + 4.7M
Q_norm/K_norm	256
MLP (gate, up, down)	25M + 25M + 25M
LayerNorm	5.1K
总计	**~60M**

36层总语言模型参数量: ~2.1B

3.4 Embedding 算子

算子	Shape	参数量
`embed_tokens.weight`	(151936, 2560)	~389M

3.5 归一化算子

RMSNorm (Root Mean Square Layer Normalization):

```
RMSNorm(x) = x / sqrt(mean(x^2) + eps) * weight
```

参数	值
`rms_norm_eps`	1e-06

---

4. 推理流程

4.1 前向传播

```
1. 图像预处理
   图像 → patchify → (B, 3, H, W) → (B, N, patch_size^2×3)

2. 视觉编码
   patch_emb → ViT (24层) → visual_features
   visual_features → linear_projector → visual_emb

3. 文本编码
   input_ids → embedding → text_emb

4. 多模态融合
   [visual_emb, text_emb] → 拼接 (含位置ID特殊处理)

5. 语言解码
```

```
   融合emb → Qwen3 Decoder (36层) → hidden_states

6. 输出预测
   hidden_states → lm_head → logits
```

4.2 生成流程

```
1. 初始化: input_ids = [BOS]
2. 循环:
   a. 前向传播得到 logits
   b. sampling (greedy/beam/sample)
   c. token = argmax(logits)
   d. input_ids += token
   e. 如果 token == EOS 或达到 max_length: break
3. return output_ids
```

---

5. 参数量统计

5.1 各组件参数量

组件	参数量	占比
Vision Tower (ViT)	~374M	~8%
Language Model	~4B	~91%
Embedding	~389M	~9%
总计	**~4.4B**	100%

5.2 模型文件

```
Qwen3-VL-4B-Instruct/
├── model-00001-of-00002.safetensors    # 4.9GB
├── model-00002-of-00002.safetensors   # 3.9GB
├── config.json                       # 模型配置
├── tokenizer_config.json              # 分词器配置
├── generation_config.json          # 生成配置
└── vocab.json                       # 词表 (151936)
```

总权重大小: ~8.7GB

---

6. 完整模型结构

```
Qwen3VLForConditionalGeneration (4.4B)
  model: Qwen3VLModel (4.0B)
    visual: Qwen2VLVisionModel (374M)
      patch_embedding: Conv2d (3, 16×16 → 768)
      position_embedding: Embedding (2304)
      blocks: ModuleList (24层)
        Block:
          norm1: RMSNorm
          attn: Attention (QKV合并)
          norm2: RMSNorm
          mlp: MLP (GELU)
      norm: RMSNorm
    language_model: Qwen3Model (2.5B)
      embed_tokens: Embedding (151936×2560)
      layers: ModuleList (36层)
        DecoderLayer:
          input_layernorm: RMSNorm
          self_attn: Qwen3Attention
            q_proj: Linear
            k_proj: Linear
            v_proj: Linear
            o_proj: Linear
            q_norm: Qwen3RMSNorm
            k_norm: Qwen3RMSNorm
          post_attention_layernorm: RMSNorm
          mlp: Qwen3MLP
            gate_proj: Linear
            up_proj: Linear
            down_proj: Linear
      norm: RMSNorm
  lm_head: Linear (2560 → 151936)
```

---

7. 关键算子公式汇总

算子	公式	PyTorch
Attention	`softmax(QK^T/√d)V`	`torch.nn.functional.scaled_dot_product_attention`
RoPE	`f(x, m) = e^(imθ)·x`	手动实现
RMSNorm	`x/√(mean(x²)+eps)·γ`	`torch.nn.functional.normalize`
SwiGLU	`silu(W₁x)·W₂x·W₃x`	`F.silu(gate) * up`
Conv2d	`y = W * x + b`	`nn.Conv2d`
Linear	`y = Wx + b`	`nn.Linear`

---

8. Qwen3-VL vs π0.5 算子对比

特性	Qwen3-VL-4B	π0.5
模型类型	VLM (多模态)	VLA (动作)
视觉编码器	ViT (24层)	SigLIP (26层)
语言模型	Qwen3 (4B)	Gemma (2B)
注意力机制	Grouped Query Attention	Multi-Head Attention
归一化	RMSNorm	AdaRMSNorm (自适应)
位置编码	RoPE + MRoPE	RoPE
动作生成	Autoregressive	Flow Matching
推理方式	Token-by-token	Multi-step sampling

---

9. 参考资料

- HuggingFace: https://huggingface.co/Qwen/Qwen3-VL-4B-Instruct
- 论文: https://arxiv.org/abs/2503.14465
- GitHub: https://github.com/QwenLM/Qwen2-VL